TUK Logo

TECHNICAL UNIVERSITY OF KENYA

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS

LINDAJAMII: A FULL-STACK COMMUNITY SAFETY AND INFORMATION PLATFORM

PRESENTED BY:

STUDENT NAME: [Your Name]

REG NO: [Your Registration Number]

SUPERVISED BY:

[Supervisor's Name]

PROPOSAL SUBMITTED TO THE SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY IN PARTIAL FULFILLMENT FOR THE BACHELOR OF / DIPLOMA IN

TECHNOLOGY IN COMPUTER TECHNOLOGY PROJECT OF THE TECHNICAL UNIVERSITY OF KENYA

SUBMISSION DATE:

14 May 2026

DECLARATION

I, the undersigned, declare that this project proposal is my original work and has not been submitted to any other university or institution for the award of a degree or diploma. All sources of information have been specifically acknowledged by means of references.

Student Name: _____

Signature: _____

Date: _____

This project proposal has been submitted for examination with my approval as the University Supervisor.

Supervisor Name: _____

Signature: _____

Date: _____

DEDICATION

This project is dedicated to the communities striving for safety, transparency, and resilience. It is also dedicated to my family and friends for their unwavering support and encouragement throughout my academic journey.

ACKNOWLEDGEMENT

I would like to express my profound gratitude to my supervisor, [Supervisor's Name], for their invaluable guidance, constructive criticism, and continuous support during the formulation of this proposal. I also extend my thanks to the faculty members of the School of Computing and Information Technology at the Technical University of Kenya for equipping me with the necessary knowledge and skills. Finally, I am grateful to my peers for their collaborative spirit and insightful discussions.

LIST OF ABBREVIATIONS

- **API:** Application Programming Interface
- **CRUD:** Create, Read, Update, Delete
- **DFD:** Data Flow Diagram
- **ERD:** Entity-Relationship Diagram
- **HTML:** HyperText Markup Language
- **HTTP:** Hypertext Transfer Protocol
- **JSON:** JavaScript Object Notation
- **JWT:** JSON Web Token
- **REST:** Representational State Transfer
- **SCIT:** School of Computing and Information Technology
- **SPA:** Single Page Application
- **SQL:** Structured Query Language
- **STK:** Sim Tool Kit
- **UI:** User Interface
- **UML:** Unified Modeling Language

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

The rapid urbanization and increasing complexity of modern societies have highlighted significant gaps in community safety and information dissemination. Traditional communication channels are often fragmented, leading to delayed incident reporting and a lack of verified, real-time information regarding local threats, weather changes, and relevant news. This project proposes the development of “LindaJamii,” a comprehensive, full-stack web platform designed to address these challenges. LindaJamii will provide a centralized hub for real-time incident reporting, community alerts, and an integrated InfoHub delivering localized weather forecasts and curated news (medical, geopolitical, climatic). Furthermore, the platform will integrate the Safaricom Daraja API to facilitate secure M-Pesa donations, empowering communities to financially support local safety initiatives. The system will be built using a robust, multi-language microservices architecture, utilizing C for high-performance utilities, Python (Flask) for incident and information APIs, and Java (Spring Boot) for secure neighbourhood and patrol management, all connected to a modern HTML/CSS/JavaScript frontend. This proposal outlines the background, objectives, methodology, and system analysis required to successfully implement LindaJamii, aiming to foster a more informed, responsive, and resilient community environment.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION

1.1 Introduction

The concept of community safety has evolved significantly with the advent of digital technology. However, many local communities still struggle with fragmented communication, delayed responses to incidents, and a lack of access to verified, real-time information. LindaJamii, a Swahili term translating to “Protect Community,” is conceptualized as a comprehensive digital platform designed to bridge these gaps. It aims to empower residents by providing a centralized hub for incident reporting, safety alerts, and critical information dissemination.

This project will involve the design and implementation of a full-stack web application. The architecture will employ a modern, distributed approach, utilizing a multi-language backend to handle specific tasks efficiently: a C-based microservice for low-level utilities, a Python (Flask) API for rapid data processing and external integrations, and a Java (Spring Boot) API for robust, secure enterprise logic. The frontend will be a responsive Single Page Application (SPA) providing an intuitive user interface. Key features will include real-time incident tracking, an InfoHub for weather and news, and an integrated M-Pesa payment gateway to facilitate community donations.

1.2 Background of the Study

1.2.1 Background of the Organization

LindaJamii is proposed as a versatile platform intended for adoption by neighborhood associations, local security groups (such as Nyumba Kumi initiatives in Kenya), and community-based organizations. Historically, these groups have relied on physical meetings, notice boards, or basic messaging apps (like WhatsApp or Telegram) to coordinate activities. While these methods have fostered community spirit, they lack the structure required for efficient incident management, data analysis, and secure financial contributions. The growth of these organizations necessitates a dedicated

system that can handle complex workflows, ensure data privacy, and provide actionable intelligence to community leaders and residents alike.

1.2.2 Overview of Existing System

Currently, community safety and information sharing are largely decentralized. Residents might use a WhatsApp group to report a suspicious person, check a separate weather app for forecasts, and rely on mainstream media for news.

Procedures in the Existing System:

1. An incident occurs.
2. A resident types a message in a community chat group.
3. The message may be missed if the chat is highly active.
4. Community leaders manually record the incident if they see it.
5. Financial contributions for community projects are collected via personal mobile money transfers, lacking transparency and automated receipting.

Disadvantages of the Existing System:

- **Information Overload and Loss:** Critical alerts are easily buried in informal chat groups.
- **Lack of Verification:** It is difficult to verify the authenticity of reports shared informally.
- **Inefficient Resource Allocation:** Without centralized data, it is challenging to identify crime hotspots or trends.
- **Financial Opacity:** Manual collection of funds lacks transparency and accountability.
- **Fragmented Information:** Residents must use multiple apps to stay informed about their environment (weather, news, local alerts).

1.2.3 Overview of the Proposed System

The proposed LindaJamii system will centralize these functions into a single, cohesive web application.

Procedures in the Proposed System:

1. A user logs into the LindaJamii dashboard.
2. To report an incident, the user fills out a structured form (category, severity, location).
3. The system processes the report, updates the central database, and immediately displays it on the community feed.
4. The InfoHub automatically fetches and displays real-time weather and relevant news.
5. Users can securely donate to community funds via an integrated M-Pesa STK push directly from the platform.

Benefits of the Proposed System:

- **Centralized Communication:** A single source of truth for community safety and information.
- **Structured Data:** Incident reports are categorized and easily analyzable.
- **Real-time Awareness:** The InfoHub keeps residents informed of environmental and geopolitical factors affecting their safety.
- **Transparent Funding:** Integrated M-Pesa gateway ensures secure, trackable donations.
- **Scalability:** The microservices architecture allows the system to grow and handle increased traffic efficiently.

1.3 Problem Statement

Despite the proliferation of communication technologies, local communities lack a dedicated, integrated platform for managing safety and disseminating critical information. The reliance on fragmented, informal channels (such as social media groups) results in delayed incident reporting, unverified information, and a lack of actionable data for community leaders. Furthermore, the absence of a transparent, integrated mechanism for community funding hinders the sustainability of local safety initiatives. This project addresses the problem of inefficient community coordination by developing LindaJamii, a unified system that streamlines incident reporting, provides real-time environmental and news updates, and facilitates secure financial contributions, thereby enhancing overall community resilience.

1.4 Objectives

1.4.1 Project Goal (Major Objective)

The primary goal of this project is to design, develop, and implement LindaJamii, a full-stack, multi-language web platform that enhances community safety and awareness through centralized incident reporting, real-time information dissemination, and integrated mobile payment capabilities.

1.4.2 General Objectives

- To develop a secure and scalable distributed backend architecture.
- To create an intuitive and responsive user interface for community members.
- To integrate external APIs to provide real-time, relevant data to users.
- To implement a secure financial transaction module for community support.

1.4.3 Specific Objectives

- To design and implement a relational database schema to manage users, incidents, patrols, and notifications.
- To develop a Java (Spring Boot) REST API with Spring Security for managing user authentication and neighborhood patrol schedules.
- To develop a Python (Flask) REST API to handle incident reporting logic and integrate external Weather and News APIs.
- To develop a C-based microservice to handle high-performance utility tasks such as data hashing and system health monitoring.
- To integrate the Safaricom Daraja API (STK Push) within the Python backend to process M-Pesa donations.
- To build a modern HTML/CSS/JavaScript frontend featuring a 5-step onboarding flow, an interactive dashboard, and dedicated components for the InfoHub and Donation gateway.

1.5 Justification

The realization of the LindaJamii project will bring significant value to local communities. By transitioning from informal communication methods to a structured platform, communities will benefit from faster response times to incidents and a clearer understanding of local safety trends. The integration of the InfoHub ensures that residents are not only aware of immediate local threats but also broader environmental and health factors that could impact them. The M-Pesa integration is particularly crucial in the Kenyan context, providing a familiar, secure, and transparent method for funding community initiatives, such as paying neighborhood watch personnel or installing streetlights. Technically, the project presents a challenging and highly relevant exercise in building distributed systems, utilizing a polyglot architecture that reflects modern enterprise software development practices.

1.6 Scope of the Study

The project will cover the end-to-end development of the LindaJamii web platform. This includes:

- **Frontend:** Development of the user interface using HTML, CSS, and JavaScript, focusing on responsive design and user experience.
- **Backend:** Implementation of three distinct services (Java, Python, C) and their inter-communication.
- **Database:** Design and implementation of a PostgreSQL/MySQL relational database.
- **Integrations:** Connection to the Safaricom Daraja API for payments, OpenWeatherMap for weather data, and NewsAPI for curated news. The scope is limited to the web application; native mobile applications (Android/iOS) will not be developed in this phase. The system will be designed to serve a localized community structure (e.g., a specific neighborhood or estate) as a proof of concept.

1.7 Limitations of the Proposed System

- **Internet Dependency:** The system requires a stable internet connection to function, which may be a constraint in areas with poor connectivity.
- **Third-Party API Reliance:** The functionality of the InfoHub and Donation gateway depends entirely on the uptime and rate limits of external providers (Safaricom, OpenWeatherMap, NewsAPI).
- **Hardware Constraints:** The initial deployment will be on a single server environment, which may limit the number of concurrent users it can handle before requiring a distributed cloud deployment.

1.8 Project Risk and Mitigation

Risk	Description	Mitigation Strategy
API Integration Failure	Changes in third-party API endpoints or authentication methods (e.g., Daraja API).	Implement robust error handling and fallback mechanisms. Regularly monitor API documentation for updates.
Security Vulnerabilities	Unauthorized access to user data or fraudulent donation requests.	Utilize Spring Security for robust authentication (JWT), bcrypt for password hashing, and validate all inputs.
Scope Creep	The project expands beyond the initial objectives, causing delays.	Strictly adhere to the defined scope and specific objectives. Any new features will be documented for future versions.
Time Constraints	Inability to complete the multi-language backend within the academic timeframe.	Utilize agile development methodologies, prioritizing core features (MVP) first before adding complex integrations.

1.9 Project Schedule

The project will be executed over a 12-week period, following an iterative development approach.

Phase	Activity	Duration
1	Requirement Gathering & System Analysis	2 Weeks
2	System Design (Architecture, Database, UI/UX)	2 Weeks
3	Backend Development (Java, Python, C)	3 Weeks
4	Frontend Development & API Integration	2 Weeks
5	External API Integration (M-Pesa, Weather, News)	1 Week
6	System Testing & Debugging	1 Week
7	Documentation & Final Presentation Preparation	1 Week

(A detailed Gantt chart will be provided in the Appendix of the final report).

1.10 Budget and Resources

This project is primarily software-based and will utilize open-source technologies, minimizing direct financial costs.

Item	Description	Estimated Cost (KES)
Hardware	Personal Computer (Core i5, 8GB RAM, 256GB SSD)	Existing
Software	IDEs (VS Code, IntelliJ), PostgreSQL, Git	Free / Open Source
Hosting	Cloud VPS (e.g., DigitalOcean, AWS Free Tier) for deployment	2,000 / month
Internet	Data bundles for research and API communication	3,000
Miscellaneous	Printing, binding of final reports	1,500
Total		**6,500**

CHAPTER TWO: LITERATURE REVIEW

2.1 Reviewed Similar Systems

Several systems exist that attempt to address community safety and communication, though often in a fragmented manner.

1. **Nextdoor:** A widely used hyper-local social networking service for neighborhoods. It allows users to share information, organize events, and report safety concerns.
2. **Citizen:** A mobile app that sends location-based safety alerts in real-time. It relies on monitoring emergency scanner radios and user-generated reports.
3. **WhatsApp/Telegram Groups:** The most common informal method used by communities in Kenya (e.g., Nyumba Kumi groups) for rapid communication and alerts.

2.2 Tools and Methodologies used in Reviewed Systems

- **Nextdoor:** Utilizes a monolithic architecture transitioning to microservices, heavily relying on geospatial data processing to verify user addresses and group them into neighborhoods. *Advantage:* High user engagement. *Disadvantage:* Prone to gossip and unverified information; lacks integrated financial tools for community projects.
- **Citizen:** Employs real-time data streaming technologies (like WebSockets or Kafka) to push alerts instantly to users. *Advantage:* Extremely fast alert dissemination. *Disadvantage:* Can cause unnecessary panic; lacks broader community building features (like news or weather).
- **WhatsApp Groups:** Uses standard instant messaging protocols. *Advantage:* Ubiquitous and easy to use. *Disadvantage:* Unstructured data, impossible to analyze trends, no integration with external APIs (like M-Pesa or Weather).

2.3 Gaps in the Existing Systems and the Proposed Solution

The primary gap identified in the reviewed systems is the lack of a **holistic, integrated approach** tailored to the specific needs of developing communities, particularly in regions like East Africa.

- **Financial Integration:** None of the reviewed safety platforms natively integrate local mobile money solutions (like M-Pesa) to fund community policing or infrastructure.
- **Contextual Information:** While apps like Citizen provide crime alerts, they do not provide contextual environmental data (weather, health news) which are equally critical to community resilience.
- **Structured Reporting vs. Accessibility:** WhatsApp is accessible but unstructured. Nextdoor is structured but often too broad.

2.4 The Proposed Solution

LindaJamii bridges these gaps by offering a tailored, full-stack solution. It provides the structured incident reporting of a dedicated app, combined with the contextual awareness of an InfoHub (Weather/News), and crucially, integrates Safaricom's Daraja API for seamless M-Pesa donations. By utilizing a microservices architecture (Java, Python, C), the system ensures that high-load tasks (like real-time alerts) do not interfere with secure transactions or data processing, providing a robust and scalable platform superior to informal chat groups.

CHAPTER THREE: METHODOLOGY

3.1 Methodology and Tools

This project will adopt the **Agile Software Development Methodology**, specifically utilizing the Scrum framework. Agile is chosen because it promotes iterative development, allowing for continuous feedback and flexibility in accommodating changes, which is crucial when integrating multiple third-party APIs (M-Pesa, Weather, News) that may require adjustments during development.

Phases of the Agile Methodology:

1. **Planning & Requirements:** Defining the product backlog based on the objectives.
2. **Design:** Architecting the multi-language backend and designing the UI/UX.
3. **Development (Sprints):** Developing features in 2-week sprints (e.g., Sprint 1: Java Backend & Auth; Sprint 2: Python API & InfoHub; Sprint 3: Frontend Integration).
4. **Testing:** Continuous integration and testing during each sprint.
5. **Deployment:** Deploying the application to a cloud environment.

3.2 Source of Data

- **Primary Data:** Data generated by users interacting with the system (incident reports, user profiles, donation records).
- **Secondary Data:** Data fetched from external APIs:
    - Weather data from OpenWeatherMap API.
    - News articles from NewsAPI.
    - Transaction status data from Safaricom Daraja API.

3.3 Data Collection Methods

For the purpose of system requirement gathering and validation:

- **Document Inspection:** Reviewing existing literature on community policing and digital safety platforms.
- **Observation:** Observing how current community groups (e.g., WhatsApp groups) handle information and financial contributions to understand user behavior and pain points.
- **Prototyping:** Developing a rapid prototype of the UI to gather feedback on usability before full implementation.

3.4 Resources Required / Materials

3.4.1 Hardware Specifications

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5
Memory (RAM)	4 GB	8 GB or higher
Storage	100 GB HDD	256 GB SSD
Internet	5 Mbps connection	20 Mbps connection

3.4.2 Software Specifications

Component	Specification
Operating System	Ubuntu Linux 22.04 (Development & Server)
Programming Languages	Java 17, Python 3.11, C11, JavaScript (ES6+)
Frameworks	Spring Boot 3.x (Java), Flask (Python)
Database	PostgreSQL 14+
Web Server	Nginx / Python HTTP Server (Frontend hosting)
Tools	Git, Maven, pip, gcc, Postman (API testing)

CHAPTER FOUR: SYSTEM ANALYSIS AND REQUIREMENT MODELING

4.1 Introduction to System Analysis

System analysis involves a detailed examination of the current methods used for community communication and the formulation of a new, integrated digital system. This chapter outlines the feasibility of LindaJamii and models the requirements using standard analysis tools to ensure the proposed system meets the defined objectives.

4.2 Objectives of System Analysis

- To determine the technical, economic, and operational feasibility of LindaJamii.
- To clearly define the functional and non-functional requirements of the system.
- To model the flow of data and system processes using Context Diagrams and Data Flow Diagrams (DFDs).

4.3 Problem Definition

The core problem is the inefficiency and fragmentation of current community safety communication. Information is siloed, incident reporting is unstructured, and community funding mechanisms are opaque. The proposed system must solve this by providing a unified interface that handles structured data entry (incidents), external data retrieval (news/weather), and secure financial transactions (M-Pesa).

4.4 Feasibility Study

- **Technical Feasibility:** The project is technically feasible. The chosen technologies (Java, Python, C, HTML/JS) are well-documented, open-source, and

widely supported. The integration of RESTful APIs is a standard industry practice.

- **Economic Feasibility:** The project is economically viable. Development relies on open-source software. The only costs incurred are for cloud hosting and internet access, which are minimal and within the student budget.
- **Operational Feasibility:** The system is designed to be user-friendly, requiring minimal training. If adopted by a community, it will streamline operations, making it highly operationally feasible.
- **Schedule Feasibility:** The 12-week timeframe is sufficient to develop the core MVP (Minimum Viable Product) using Agile methodologies, provided scope creep is managed.

4.5 System Analysis Tools

The following tools will be used to model the system:

- **Context Diagrams:** To show the system boundaries and interactions with external entities (Users, M-Pesa API, Weather API).
- **Data Flow Diagrams (DFDs):** To illustrate how data moves through the system processes.
- **Entity-Relationship Diagrams (ERDs):** To model the database structure.

4.6 System Investigation

4.6.1 Introduction

Investigation involves understanding the exact data points required to make the system functional.

4.6.2 Data Collection

Data regarding required features was collected by analyzing the shortcomings of existing platforms (like WhatsApp groups) where messages lack structure (e.g., missing location data for an incident).

4.6.3 Fact Recording

Program Requirements:

- **Input Requirements:** User registration details, incident details (type, location, description), donation amount, phone number.
- **Output Requirements:** Dashboard feed of incidents, weather forecast display, news list, payment success/failure notifications.
- **Process Requirements:** User authentication (JWT), routing requests to appropriate microservices, initiating STK push, fetching external API data.

4.7 System Analysis

The analysis of the requirements dictates a distributed architecture.

Data Flow Representation:

1. **User Entity** interacts with the **Frontend UI**.
2. Frontend sends authentication requests to the **Java API**.
3. Frontend sends incident reports and external data requests to the **Python API**.
4. The Python API communicates with the **M-Pesa API** for donations and **OpenWeather/NewsAPI** for the InfoHub.
5. The Python API may call the **C Microservice** for specific high-speed data hashing or validation tasks.

(Note: Detailed Context Diagrams and DFDs will be included in the final project documentation).

BIBLIOGRAPHY

1. Sommerville, I. (2015). *Software Engineering*. 10th ed. Pearson.
2. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
3. Safaricom Developer Portal. (2023). *Daraja API Documentation*. [Online] Available at: https://developer.safaricom.co.ke/ [Accessed 14 May 2026].
4. OpenWeatherMap. (2023). *Weather API Documentation*. [Online] Available at: https://openweathermap.org/api [Accessed 14 May 2026].
5. NewsAPI. (2023). *News API Documentation*. [Online] Available at: https://newsapi.org/docs [Accessed 14 May 2026].
6. Spring Framework Documentation. (2023). *Spring Boot Reference Guide*. [Online] Available at: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ [Accessed 14 May 2026].
7. Pallets Projects. (2023). *Flask Documentation*. [Online] Available at: https://flask.palletsprojects.com/ [Accessed 14 May 2026].